AGENTIC AI FRAMEWORKS

Agentic AI — LangChain, LangGraph, CrewAI, AutoGen, OpenClaw, NemoClaw, ZeroClaw Pros, cons, and what nobody tells you

What makes something an agent

An LLM responds. An agent perceives, plans, acts, and loops. The difference is autonomy across multiple steps — using tools, maintaining memory, making decisions without a human in the loop at every turn. The framework you choose determines how much of that autonomy you actually get, and at what cost.

LangChain

The most widely used starting point for building LLM-powered applications. It works by chaining sequences of LLM calls together with tools, memory, and external data sources.

Pros: massive ecosystem, excellent documentation, huge community support, great for RAG pipelines and document workflows.

Cons: abstraction hell at scale. Complex flows become unmaintainable chains of chains. It was not designed for true multi-agent coordination and starts showing cracks the moment you need agents to coordinate dynamically rather than follow a fixed sequence.

LangGraph

LangChain's architectural evolution. Replaces linear chains with a graph structure — agents as nodes, state transitions as edges. Supports loops, conditionals, and parallel branches that LangChain simply cannot handle cleanly.

Pros: handles complex multi-step reasoning, properly stateful, supports cycles and conditional flows, much closer to how real agent behavior works.

Cons: significantly steeper learning curve, debugging graph state across multiple nodes is painful, still assumes a digital environment and does not address physical deployment.

CrewAI

Role-based multi-agent framework. You define agents as crew members, each with a role, a goal, and a backstory. They collaborate like a team to complete a shared objective.

Pros: intuitive mental model, fast to prototype, works well for research and document-heavy workflows, easy for non-experts to understand.

Cons: limited control over internals, agent behavior can be unpredictable when tasks get ambiguous, the backstory-based prompting approach feels fragile at production scale.

AutoGen (Microsoft)

Conversation-driven multi-agent framework. Agents communicate with each other through dialogue — a user agent, an assistant agent, and optionally a critic agent debate until they converge on a solution.

Pros: excellent for code generation and adversarial reasoning tasks, built-in support for human-in-the-loop interactions, Microsoft-backed with strong research output.

Cons: conversations become verbose and expensive quickly, hard to control output format, not deployment-ready out of the box, better suited for research than production pipelines.

OpenClaw, NemoClaw, ZeroClaw

A family of more opinionated, deployment-focused frameworks building on top of the foundations above. ZeroClaw is what the program is actively building this week. These frameworks prioritize structured workflows and real-world deployment over research flexibility.

Pros: closer to production-grade, more structured and predictable, deployment context is a first-class consideration rather than an afterthought.

Cons: smaller communities, documentation still maturing, inherit the underlying limitations of the frameworks they are built on, and the physical AI problem remains unsolved even here.

Honest comparison

LangChain is the best entry point but the worst at scale. LangGraph solves the state problem but not the deployment problem. CrewAI is the most intuitive but the least predictable. AutoGen is the most rigorous but the most expensive. ZeroClaw is the most deployment-aware but still maturing.

None of them were built for physical AI. No native cybersecurity layer, no audit trail by default, no embodied deployment support. They all assume a digital environment, a reliable connection, and a forgiving failure mode. None of those assumptions hold in robotics or critical infrastructure.

Reasoning

I went through each framework not to list features but to ask one question: where does it break? LangChain breaks at scale. LangGraph breaks at debugging. CrewAI breaks at unpredictability. AutoGen breaks at cost. The pattern is that every framework optimizes for the demo, not the deployment. ZeroClaw and the OpenClaw family are interesting precisely because they are trying to solve the deployment problem — but they are doing it on top of the same foundations that carry the same gaps. The real question nobody is asking: what does a framework look like that was designed for physical AI from day one, with security and audit as core primitives and not plugins?

Taxonomy of Agentic AI Systems How to classify what you are actually building

Why taxonomy matters

Before you build or evaluate an agent system, you need to know what kind of system it is. Taxonomy gives you a shared language so that when someone says multi-agent system, everyone in the room means the same thing. Without it, you cannot evaluate, secure, or govern what you are building.

Dimension 1 — By coordination style

Single agent: one LLM, one tool set, one task loop. Simple and controllable but fundamentally limited in what it can accomplish. Examples: basic LangChain agent, simple AutoGen assistant.

Hierarchical MAS: a manager agent decomposes tasks and delegates to specialist worker agents. The most common production pattern today. Examples: CrewAI, ZeroClaw, AXON NEURA.

Conversational MAS: agents negotiate through dialogue with no fixed hierarchy. Consensus emerges from debate rather than delegation. Examples: AutoGen multi-agent conversations, NemoClaw.

Swarm and decentralized: no manager, no fixed roles. Emergent behavior from simple local rules applied across many agents. Powerful but extremely difficult to control or audit. Currently research-stage for most applications.

Dimension 2 — By autonomy level

L1 Simple reflex: if X then Y. No memory, no planning, purely reactive.

L2 Rule plus memory: stateful, follows scripted logic, remembers context within a session.

L3 Goal-driven: plans across multiple steps, uses tools, reasons using an LLM backbone. This is where most current frameworks operate.

L4 Self-improving: learns from feedback, adapts behavior over time, can fine-tune or modify its own approach.

L5 Full autonomy: multi-agent coordination with no human in the loop. True L5 systems are rare and carry serious governance implications.

The autonomy level is the most important dimension for safety. An L3 agent making mistakes is a user experience problem. An L5 agent making mistakes on physical infrastructure is a liability problem.

Dimension 3 — By deployment context

Digital only: web tasks, code generation, document analysis. All major frameworks live here. Failure modes are forgiving — a wrong answer is annoying, not dangerous.

Edge plus cloud hybrid: IoT devices, telecom CPE, smart infrastructure. Agents split their work between edge compute and cloud processing. AXON NEURA operates in this space.

Embodied and physical: robots, drones, rovers, factory systems. Real-world consequences. No framework today handles this natively or well. This is the frontier.

The gap nobody is mapping

Current taxonomy stops at digital. The moment you add a physical actuator — a robot arm, a rover, a drone — the entire risk profile changes. A bad decision in a digital agent produces a bad output. A bad decision in an embodied agent produces a physical event. The taxonomy needs a fourth dimension: consequence severity. Ranging from annoying at one end, meaning a wrong answer, to catastrophic at the other, meaning physical harm or infrastructure failure. No current framework classifies agents this way and that omission is a serious blind spot.

Reasoning

Taxonomy is not academic — it is practical. If you cannot classify what you are building, you cannot evaluate it, secure it, or govern it. The autonomy level dimension is the most important one for safety, and the deployment context dimension is the one the entire industry is ignoring. Everyone is building for digital environments and assuming that embodied AI is just digital plus a robot. It is not. The physics of the real world — latency, sensor noise, actuator failure, irreversibility of actions — changes everything about how you design agent behavior and failure modes. A framework that does not account for deployment context is not a complete framework. It is a prototype.

Ontology of Multi-Agent AI Systems Not what things are — how they connect

Taxonomy vs ontology — the actual difference

Taxonomy classifies. Ontology maps relationships. Taxonomy tells you this is a hierarchical MAS. Ontology tells you that a manager agent HAS worker agents, USES shared memory, OPERATES IN an environment, and LOGS TO an audit layer. The relationships are where the accountability lives, and they are what most people skip entirely.

The core ontology

Agent has memory, tools, and an LLM backbone. Agent perceives environment state. Agent communicates with other agents. Agent executes actions. Agent logs to audit layer.

Environment has state. Environment provides observations to agents. Environment receives actions from agents. Environment can be digital, physical, or hybrid.

Audit layer records agent actions and reasoning. Audit layer enforces governance policy. Audit layer enables human oversight. Audit layer provides explainability when something goes wrong.

Governance policy constrains agent behavior. Governance policy defines the permitted action space. Governance policy escalates uncertain or high-risk decisions to humans.

Why each relationship matters

Agent logs to audit layer: this is the relationship that almost no framework implements natively. Without it you have no traceability. You cannot answer why the agent did that. This is not optional in production, especially in regulated or physical environments.

Environment can be physical: the moment this holds, every other relationship changes in kind not just degree. Latency matters. Sensor noise matters. Actuation failure matters. Irreversibility matters. A digital environment tolerates mistakes. A physical environment does not.

Governance policy escalates to humans: this is the missing link in most current frameworks. Agents need to know when they do not know. Escalation logic — if confidence is below threshold or action consequence exceeds defined risk level, pause and involve a human — should be a first-class ontological concept and not something bolted on afterward.

Applied to Chakra47 OS

Perception layer observes the physical environment. World model maintains a live representation of environment state. Reasoning plus audit layer decides what to do and logs every decision with its rationale. Agentic execution acts on the physical world. ROS2 connects all layers through the DDS communication bus.

This is what a properly grounded ontology for physical AI looks like. Every relationship is explicit. Every layer is accountable. The audit relationship is not optional — it is baked into the architecture at layer three.

The critical insight

Most agentic AI ontologies stop at the agent boundary. They define what agents do but not what constrains them. A complete ontology must include governance, audit, and escalation as first-class entities and not optional add-ons. This is the difference between an agent system that is powerful and one that is trustworthy. Power without accountability is just a liability waiting to materialize.

The other missing piece is the physical environment relationship. Every ontology in every major framework assumes the environment is digital and forgiving. The moment you deploy into the physical world that assumption breaks and the entire ontology needs to be rebuilt around it. Nobody in the mainstream agentic AI space has done this yet. That is the gap Chakra47 is designed to fill.

Reasoning

Ontology sounds abstract but it has a very concrete payoff: if you can draw the relationships, you can find the gaps. Every missing relationship in an ontology is a missing accountability in the system. When I mapped the standard agentic AI ontology, the gap that immediately stood out was the missing connection between the agent and any audit or governance layer. Agents act, but nobody records why. In a digital chatbot that is acceptable. In a system controlling physical infrastructure or autonomous vehicles, that missing relationship is a liability that could end a company. Chakra47 is essentially a complete ontology made into software — every relationship mapped above exists as an explicit architectural layer in the stack. That is not a coincidence. That is what it means to design for accountability from the beginning.